

TOPIC 01: DATABASE & SQL FUNDAMENTALS

Developer, SQL & MySQL Communication Pipeline:

How developers interact with database storage via standard query architecture:



What is a Database? (Core Definition)

A Database is an **organized collection of structured data** stored and formatted in a computer system so that it can be **efficiently accessed**, queried, managed, and updated electronically.

Why Use a Database? (Key Advantages)

- **Massive Storage Capacity:** Handles gigabytes to terabytes without performance degradation.
- **Granular Security & Access Control:** Enforces user authentication and role-based permissions.
- **Scalability & Query Optimization:** Indexes allow sub-millisecond retrieval across millions of records.
- **Streamlined CRUD Operations:** Inserting, updating, and deleting records is structured and safe.
- **ACID Compliance:** Ensures transaction atomicity, consistency, isolation, and durability against crashes.

 **Core Distinction:** MySQL is a **Relational Database Management System (RDBMS)** software, whereas SQL is the **Structured Query Language** used to communicate with that RDBMS!

TOPIC 02: SQL vs NoSQL (THE MASTER COMPARISON)

Relational (SQL) vs Non-Relational (NoSQL) Databases:

Feature / Parameter	SQL (Relational Databases)	NoSQL (Non-Relational Databases)
Data Model	Structured Tables with Rows & Columns.	Documents (JSON) , Key-Value, Columns, Graphs.
Schema	Strict / Predefined Schema: Columns & types must be declared upfront.	Dynamic / Flexible Schema: Records can have varying fields and shapes.
Scalability	Vertical Scaling: Increased by upgrading hardware (RAM/CPU).	Horizontal Scaling: Scaled by adding low-cost cluster nodes (Sharding).
Relationships & Joins	Supports complex table relationships and JOIN queries.	Little to no native JOIN support; data is typically denormalized.
Primary Strengths	Complex queries, transactions, banking, e-commerce (ACID).	Big data, real-time analytics, rapid prototyping, social feeds.
Industry Examples	MySQL, PostgreSQL, Oracle, SQLite, SQL Server	MongoDB, Cassandra, Redis, Neo4j, CouchDB



Formal Terminology: In relational database theory, a Table is termed a **Relation**, a Row is termed a **Tuple**, and a Column is termed an **Attribute**!

TOPIC 03: RDBMS DESIGN & TOPIC 04: DATABASE QUERIES

Topic 03: Relational Table Architecture & Schema

- **Columns (Attributes / Schema Design):** Defines the structural fields (e.g., id, name, email, marks).
- **Rows (Tuples / Records):** Represents a single, distinct data entry instance (e.g. (101, 'Shivam', 'shivam@gmail.com', 95)).

Topic 04: Database Administration Queries (CRUD on Database)

Standard commands for managing database containers:

```
database-queries.sql

-- 1. Create a new Database
CREATE DATABASE college;
CREATE DATABASE IF NOT EXISTS college; -- Prevents error if already present

-- 2. List all accessible databases on the server
SHOW DATABASES;

-- 3. Select and switch context to specific database
USE college;

-- 4. Display all tables in the active database
SHOW TABLES;

-- 5. Delete Database and all its internal schemas/data
DROP DATABASE college;
DROP DATABASE IF EXISTS college;
```



Caution: DROP DATABASE permanently destroys the database and all encapsulated tables and data without rollback!

TOPIC 05: CREATE TABLE & MYSQL DATA TYPES

Create Table Statement:

```
CREATE TABLE student (  
  id INT PRIMARY KEY,  
  name VARCHAR(50) NOT NULL,  
  age INT CHECK (age ≥ 18),  
  city VARCHAR(30) DEFAULT 'Delhi'  
);
```

create-table.sql

Standard MySQL Data Types:

- **INT / INTEGER:** Standard 4-byte integer values.
- **TINYINT:** 1-byte integer (-128 to 127); frequently utilized as a boolean flag (0 or 1).
- **CHAR(n):** Fixed-length character string (e.g. CHAR(10) allocating 'abc' still reserves 10 bytes). Faster lookup.
- **VARCHAR(n):** Variable-length character string (e.g. VARCHAR(50) allocating 'abc' consumes only 3 bytes). Space-efficient!
- **BLOB (Binary Large Object):** Stores binary media such as images, audio files, and PDFs.
- **DATE & DATETIME:** Temporal fields; DATE (YYYY-MM-DD) and DATETIME (YYYY-MM-DD HH:MM:SS).



CHAR vs VARCHAR: For values with deterministic static lengths (such as postal PIN codes or country codes), select CHAR. For dynamic text (names, emails), choose VARCHAR!

TOPIC 06: SQL CONSTRAINTS (DATA INTEGRITY RULES)

The 6 Core SQL Integrity Constraints:

Constraints enforce domain integrity and prevent corrupt or invalid data from entering tables:

- **1. NOT NULL:** Guarantees that a column cannot store NULL or empty values.
`name VARCHAR(50) NOT NULL`
- **2. UNIQUE:** Ensures that all values in a designated column are completely distinct.
`email VARCHAR(100) UNIQUE`
- **3. DEFAULT:** Assigns an automated fallback value when no value is provided during insertion.
`city VARCHAR(30) DEFAULT 'Delhi', salary INT DEFAULT 25000`
- **4. CHECK:** Validates that all inserted values satisfy a specified Boolean predicate.
`CONSTRAINT age_check CHECK (age >= 18 AND city = 'Delhi')`
- **5. PRIMARY KEY:** Uniquely identifies each record in a table. It is an immutable combination of **NOT NULL + UNIQUE**. Exactly **ONE** Primary Key is allowed per table.
`id INT PRIMARY KEY` or `PRIMARY KEY (id)`
- **6. FOREIGN KEY:** Prevents actions that would destroy links between tables, enforcing referential integrity.
`FOREIGN KEY (cust_id) REFERENCES customer(id)`



Key Relationship: The Primary Key uniquely identifies a record in the parent table; the Foreign Key in the child table references that exact Primary Key!

TOPIC 07: INSERT & SELECT & TOPIC 08: WHERE CLAUSE

Topic 07: Data Insertion & Retrieval (Basic DML/DQL)

```
insert-and-select.sql

-- Insert multiple records in a single statement
INSERT INTO student (id, name, age, city)
VALUES
  (101, 'Shivam', 21, 'Delhi'),
  (102, 'Rahul', 20, 'Mumbai'),
  (103, 'Anjali', 22, 'Delhi');

-- Query all columns and rows
SELECT * FROM student;

-- Query selected columns
SELECT name, city FROM student;
```

Topic 08: WHERE Clause & Conditional Operators

The WHERE clause filters records based on specified search criteria:

- **Arithmetic:** +, -, *, /, % (e.g. marks + 5)
- **Comparison:** =, !=, >, >=, <, <= (e.g. WHERE age >= 18)
- **Logical Operators:**
 - AND: All conditions must evaluate to true (WHERE age > 18 AND city = 'Delhi').
 - OR: At least one condition evaluates to true (WHERE city = 'Delhi' OR city = 'Mumbai').
 - BETWEEN: Inclusive boundary evaluation (WHERE marks BETWEEN 80 AND 95).
 - IN: Matches against an enumerated set (WHERE city IN ('Delhi', 'Pune', 'Goa')).
 - NOT: Inverts Boolean condition (WHERE city NOT IN ('Delhi')).
 - LIKE: Pattern matching (WHERE name LIKE 'S%' - prefix search).

TOPIC 09: LIMIT, ORDER BY & TOPIC 10: AGGREGATE FUNCTIONS

Topic 09: LIMIT & ORDER BY Clauses

- **LIMIT Clause:** Constrains the total quantity of records returned:
`SELECT * FROM student LIMIT 3;` (Fetches top 3 rows)
`SELECT * FROM student LIMIT 2, 3;` (Skips offset 2, fetches subsequent 3 rows - used for Pagination)
- **ORDER BY Clause:** Sorts query output ascending (ASC - default) or descending (DESC):
`SELECT * FROM student ORDER BY age ASC;`
`SELECT * FROM student ORDER BY marks DESC;`

Topic 10: Aggregate Functions in SQL

Aggregate functions perform calculations across multiple row values to produce a single summarized metric:

Function	Purpose & Analytical Role	Example Query
COUNT()	Counts total rows or non-null column values	<code>SELECT COUNT(*) FROM student;</code>
MAX()	Computes the highest / maximum value	<code>SELECT MAX(marks) FROM student;</code>
MIN()	Computes the lowest / minimum value	<code>SELECT MIN(age) FROM student;</code>
SUM()	Calculates the mathematical summation	<code>SELECT SUM(salary) FROM employee;</code>
AVG()	Calculates the arithmetic mean	<code>SELECT AVG(marks) FROM student;</code>

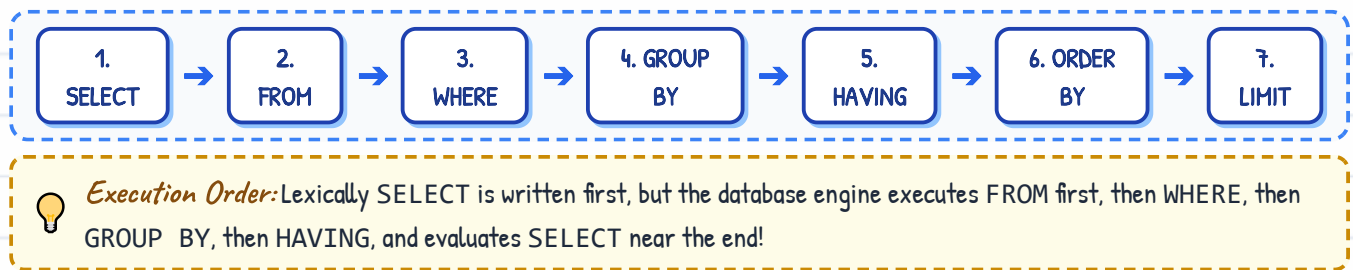
TOPIC 11: GROUP BY, HAVING & TOPIC 12: CLAUSE ORDER

Topic 11: GROUP BY & HAVING Clauses

- **GROUP BY Clause:** Groups rows sharing identical attribute values into summary rows, routinely paired with aggregate functions:
`SELECT city, COUNT(id) FROM student GROUP BY city;`
- **HAVING Clause:** Enforces conditional filters on grouped result sets.
Why not WHERE? Because WHERE applies strictly to individual base rows before grouping and cannot evaluate aggregate expressions (e.g., `COUNT(*) > 2`)!
`SELECT city, COUNT(id) FROM student GROUP BY city HAVING COUNT(id) > 2;`

Topic 12: Canonical Clause Syntax vs Execution Order Blueprint

Lexical syntax order versus internal query engine execution sequence:



TOPIC 13: UPDATE, DELETE & TOPIC 14: ALTER TABLE

Topic 13: Update & Delete Operations (DML)

```
update-and-delete.sql

-- 1. UPDATE: Mutate existing record values
UPDATE student
SET city = 'Pune', marks = 92
WHERE id = 102;

-- 2. DELETE: Remove targeted rows
DELETE FROM student
WHERE marks < 33; -- Never omit the WHERE condition!
```

Topic 14: ALTER TABLE (Schema Structure Modification)

DDL statements used to alter table attributes, constraints, and metadata:

- **Add Column:** ALTER TABLE student ADD COLUMN grade VARCHAR(2);
- **Drop Column:** ALTER TABLE student DROP COLUMN grade;
- **Rename Table:** ALTER TABLE student RENAME TO college_students;
- **Change Column (Rename + Modify Type):**
ALTER TABLE student CHANGE COLUMN age student_age INT NOT NULL;
- **Modify Column (Change Type / Constraints Only):**
ALTER TABLE student MODIFY student_age TINYINT DEFAULT 18;

TOPIC 15: TRUNCATE vs DROP vs DELETE (THE COMPARISON)

The Definitive Architectural Comparison:

Feature	DELETE	TRUNCATE	DROP
Command Classification	DML (Data Manipulation)	DDL (Data Definition)	DDL (Data Definition)
Scope of Deletion	Specified rows based on WHERE clause.	All table records / rows.	All data + Complete Table Schema.
Table Schema State	Preserved intact.	Preserved intact.	Completely destroyed and removed from database!
WHERE Filtering	Supported (WHERE id = 101)	Not permitted	Not permitted
Performance & Logging	Slower (logs row by row).	High performance (deallocates pages).	Immediate (complete schema removal).

Complete Working SQL Script (VS Code Style):

```
complete-master-queries.sql

CREATE DATABASE IF NOT EXISTS school_db;
USE school_db;

CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  marks INT CHECK (marks ≥ 0 AND marks ≤ 100),
  city VARCHAR(30) DEFAULT 'Delhi'
);

INSERT INTO students VALUES
(1, 'Aman', 85, 'Delhi'), (2, 'Shradha', 95, 'Mumbai'), (3, 'Karan', 72, 'Delhi');

-- Aggregate grouping query with HAVING and ORDER BY
SELECT city, AVG(marks) AS avg_marks FROM students
GROUP BY city HAVING AVG(marks) > 75
ORDER BY avg_marks DESC;
```